SMARTTEXT AI

Intelligent Natural Language Processing Platform

Final-Year Computer Science Project Report

Python • NLTK • spaCy • Scikit-learn • Streamlit • SQLite

Student Name: _____

College / University: _____

Department: Computer Science & Engineering

Academic Year: _____

1. Abstract

SmartText AI is a Python-based Natural Language Processing platform for analyzing written text. It combines sentiment analysis, named entity recognition, keyword extraction, extractive summarization, text classification, and text preprocessing in one Streamlit application. Results can be stored locally in SQLite and exported for further review. The project demonstrates practical language processing, machine learning, database integration, and interactive application development.

Prototype note: The demonstration classifier uses a small sample dataset, sentiment analysis uses VADER, and summarization is extractive. The system is an educational prototype, not a validated production or high-stakes decision tool.

2. Introduction and Problem Statement

Text is widely used in reviews, news, reports, feedback, and online communication. Extracting useful information manually can be time-consuming. Natural Language Processing provides computational techniques to process human language. SmartText AI brings several common NLP operations into one interface so users can inspect the tone, entities, key terms, topic, and condensed content of a text document.

3. Project Objectives

• Build an interactive NLP application using Python.

• Identify positive, negative, or neutral sentiment.

• Recognize named entities such as people, organizations, and locations.

• Extract important terms using TF-IDF.

• Produce an extractive summary of longer text.

• Classify text into demonstration topic categories.

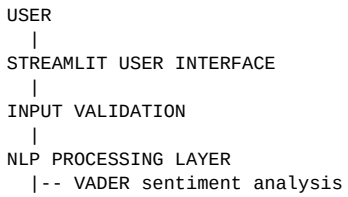
• Preprocess text using tokenization, stop-word filtering, and lemmatization.

• Store analysis history in SQLite and export results.

4. Technology Stack

Technology	Purpose
Python	Core programming language
Streamlit	Interactive web interface
NLTK / VADER	Lexicon-based sentiment analysis
spaCy	Linguistic processing and named entity recognition
Scikit-learn	TF-IDF and Logistic Regression classifier
Pandas	Data handling and history display
SQLite	Local database
Git / GitHub	Version control and source hosting

5. System Architecture



```
  |-- spaCy named entity recognition
  |-- TF-IDF keyword extraction
  |-- Extractive summarization
  |-- Scikit-learn classification
  |-- Text preprocessing
  |
RESULTS DASHBOARD
  |-- SQLite history
  |-- JSON / CSV export
```

The UI accepts text and settings. Processing functions return structured results. The application displays these results and saves an analysis record in SQLite. Each NLP component can be upgraded independently.

6. Functional Modules

6.1 Sentiment Analysis

VADER returns negative, neutral, positive, and compound scores. The prototype labels compound scores of at least 0.05 as Positive, at most -0.05 as Negative, and values in between as Neutral. Sarcasm, context, and domain-specific language can reduce reliability.

6.2 Named Entity Recognition

spaCy's English pipeline identifies spans such as PERSON, ORG, GPE, DATE, and MONEY. Predictions may be incomplete or incorrect, especially for unfamiliar names or specialized text.

6.3 Keyword Extraction

TF-IDF represents terms by their importance in a document. The prototype uses unigrams and bigrams, removes common English stop words, and returns the highest-scoring terms. For one short document, scores are only a simple ranking.

6.4 Extractive Summarization

The summarizer splits text into sentences, calculates word frequencies after stop-word removal, scores sentences, and selects the highest-scoring original sentences while preserving their order. It does not generate new wording.

6.5 Text Classification

A Scikit-learn pipeline converts text into TF-IDF features and trains Logistic Regression on a small example dataset with Technology, Sports, Business, and Education labels. This demonstrates supervised learning, but the dataset is too small for reliable generalization.

6.6 Text Preprocessing

spaCy tokenization and lemmatization are used to lowercase words, remove stop words, and filter non-alphabetic tokens. Preprocessing choices should be selected for the specific task.

7. Database Design

SQLite stores analysis records in a table named analysis_history:

```
CREATE TABLE analysis_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    input_text TEXT NOT NULL,
    sentiment TEXT,
    category TEXT,
    keywords TEXT,
    summary TEXT,
    created_at TEXT NOT NULL
);
```

The keywords field can store a JSON-encoded list. Use parameterized SQL queries for inserts. For deployment, add access controls, retention rules, and appropriate data protection.

8. MacBook Installation

Prerequisites: macOS, Python 3, and Terminal. Run:

```
mkdir SmartTextAI
cd SmartTextAI
```

```
python3 -m venv .venv
source .venv/bin/activate
mkdir -p data
touch app.py requirements.txt README.md .gitignore
python -m pip install --upgrade pip
```

Add these lines to requirements.txt:

```
streamlit
nltk
spacy
scikit-learn
pandas
```

Install packages and language resources:

```
pip install -r requirements.txt
python -m spacy download en_core_web_sm
python -c "import nltk; nltk.download('vader_lexicon')"
```

Start the application:

```
streamlit run app.py
```

Open http://localhost:8501 in your browser. Keep the virtual environment active while running the app.

9. Core Source-Code Reference

The complete application code is maintained in app.py. These excerpts illustrate the core NLP functions; the full app also requires UI navigation, validation, database operations, and export handling.

9.1 Sentiment Analysis

```
from nltk.sentiment import SentimentIntensityAnalyzer

analyzer = SentimentIntensityAnalyzer()

def analyze_sentiment(text):
    scores = analyzer.polarity_scores(text)
    compound = scores["compound"]
    if compound >= 0.05:
        label = "Positive"
    elif compound <= -0.05:
        label = "Negative"
    else:
        label = "Neutral"
    return {"label": label, "scores": scores}
```

9.2 Named Entity Recognition

```
import spacy
nlp = spacy.load("en_core_web_sm")

def extract_entities(text):
    doc = nlp(text)
    return [
        {"text": ent.text, "label": ent.label_}
        for ent in doc.ents
    ]
```

9.3 TF-IDF Keywords

```
from sklearn.feature_extraction.text import TfidfVectorizer

def extract_keywords(text, limit=8):
    vectorizer = TfidfVectorizer(
        stop_words="english",
        ngram_range=(1, 2),
        max_features=100
    )
    matrix = vectorizer.fit_transform([text])
    terms = vectorizer.get_feature_names_out()
    scores = matrix.toarray()[0]
    ranked = sorted(zip(terms, scores),
                    key=lambda x: x[1], reverse=True)
    return [term for term, score in ranked[:limit] if score > 0]
```

9.4 Classification Pipeline

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("tfidf", TfidfVectorizer(stop_words="english")),
    ("classifier", LogisticRegression(max_iter=1000))
])
model.fit(training_texts, training_labels)
prediction = model.predict([new_text])[0]
```

10. Testing Plan

Test	Action	Expected result
Empty input	Submit blank text	Validation warning
Sentiment	Submit clearly positive review	Usually Positive
NER	Submit text with known people/places	Entities shown if recognized
Keywords	Submit multi-sentence text	Ranked terms shown
Summary	Submit longer text	Selected original sentences shown
Classification	Submit supported topic text	One demo category predicted
History	Run analysis, open History	Saved record appears
Export	Download report	JSON or CSV file downloads

Do not invent accuracy figures. For meaningful evaluation, use a representative labelled dataset, held-out test data, and report accuracy, precision, recall, F1-score, and a confusion matrix.

11. GitHub Deployment

Create an empty GitHub repository named SmartTextAI. Add this .gitignore:

```
.venv/
__pycache__/
*.pyc
data/*.db
.env
.DS_Store
```

Initialize and push the project:

```
git init
git add .
git commit -m "Initial SmartText AI NLP project"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/SmartTextAI.git
git push -u origin main
```

Replace YOUR_USERNAME with your GitHub username. Do not commit private user text, local databases containing sensitive data, virtual environments, or secrets.

12. Future Enhancements

• Train the classifier on a larger, balanced, labelled dataset.

• Evaluate models using train/test splits and cross-validation.

• Add transformer models such as BERT for context-aware tasks.

• Add multilingual NLP with appropriate language-specific models.

• Build a FastAPI backend and React frontend.

• Add authentication, privacy controls, and automated tests.

• Containerize with Docker and deploy to a cloud platform.

13. Advantages

• Combines several NLP functions in one interface.

• Uses established Python libraries.

• Stores results locally and supports exports.

• Provides a foundation for a larger full-stack project.

14. Limitations and Ethical Considerations

• Models can be incorrect or biased; human review remains important.

• The demonstration classifier is not production-grade.

• The default language model is English-focused.

• Avoid submitting sensitive personal information without a clear privacy plan.

• Use datasets only when their licenses and terms permit the intended use.

15. Conclusion

SmartText AI demonstrates how Python libraries can be combined into an accessible NLP platform. It integrates sentiment analysis, named entity recognition, keyword extraction, extractive summarization, text classification, preprocessing, and local persistence. The prototype can be extended with stronger datasets, rigorous evaluation, transformer models, testing, and deployment.

16. References

• NLTK documentation: https://www.nltk.org/

• spaCy documentation: https://spacy.io/usage

• Scikit-learn documentation: https://scikit-learn.org/stable/

• Streamlit documentation: https://docs.streamlit.io/

• Pandas documentation: https://pandas.pydata.org/docs/

• Hutto, C. J. and Gilbert, E. E. (2014), VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text.